

Evaluation of IPC through Linux Pipes

Patrick DeCabooter and Shaw Liu

1 Introduction

2 Clock Precision Evaluation

Design

We tested two types of timers for our experiments. The first timer, provided by the header `time.h`, is `clock_gettime`. We chose `CLOCK_MONOTONIC` as the clock to use for `clock_gettime` due to it having guaranteed monotonicity and being system-wide, this provides nanosecond precision. The second timer, provided by the header `sys/time.h`, is `gettimeofday`. `gettimeofday` finds the time by measuring the clock cycles since the last kernel epoch, this provides microsecond precision. `gettimeofday` is a much older clock. By the time `cpu-cycles` were able to measure nanoseconds, `gettimeofday`'s interface was already locked-in. `clock_gettime` was introduced as the api to replace `gettimeofday`. In order to have similar units, we scaled `gettimeofday` up to nanoseconds instead of scaling `clock_gettime` down to microseconds. We chose to do it this way since it would take advantage of `clock_gettime`'s increased accuracy and give us more intuitive num-

bers for figures.

To evaluate the accuracy of these two timers, we decided to measure accuracy against `sleep` from the header `unistd.h` and measure the smallest measurable time. To measure against `sleep`, we started the timer, called `sleep(5)`, then measured the final time to get the difference. We did this five times in a loop for each timer, and took the minimum result. To measure the smallest measurable time, we chose to see

Results

3 Pipe Latency Evaluation

Design

Results

4 Pipe Throughput Evaluation

Design

Results

5 Conclusion

